



AGENCIA ESPACIAL

Space Mission Manager

Memoria del Proyecto

Programación · DAM · Curso 2025–2026
I.E.S. Delgado Hernández · Bollullos Par del Condado

Nombre	Rol
Javier Fernández	Jefe de Proyecto
Héctor Rodríguez	Diseñador de Base de Datos
Carlos Fernández	Desarrollador Backend
Antonio Manuel Rodríguez	Desarrollador de Interfaz
Rafa Navarro	Responsable de Calidad y Documentación

Mayo 2026

ÍNDICE

1. Descripción del proyecto	3
2. Distribución de roles	3
3. Diagrama Entidad-Relación	4
3.1 Diagrama conceptual	4
3.2 Descripción de entidades y relaciones	4
3.3 Decisiones de diseño	5
Entidad asociativa Participacion	5
Entidad débil RegistroTelemetry	5
Relación N:M EstacionSeguimiento — Satellite	5
Unicidad de nombre de misión	5
4. Diagrama de clases	6
4.1 Descripción de la arquitectura	6
4.2 Clases principales	6
Entidades (model)	6
Repositorios (repository)	6
Servicios (service)	6
Controladores UI (ui)	6
Utilidades (util)	6
5. Justificación de tecnologías	7
5.1 Java 17	7
5.2 JavaFX 17	7
5.3 ObjectDB y JPA 2.2	8
Base de datos orientada a objetos vs. relacional	8
5.4 Apache Maven	8
5.5 IntelliJ IDEA	9
6. Estrategia de Git	10
6.1 Modelo de ramas	10
6.2 Flujo de trabajo	10
6.3 Convención de mensajes de commit	10
6.4 Problemas encontrados y soluciones	11
Conflicto de merge en persistence.xml	11
Rama feature/ui retrasada respecto a main	11
7. Estructura del proyecto	12
7.1 Descripción de los paquetes	13

1. Descripción del proyecto

Agencia Espacial — Space Mission Manager es una aplicación de escritorio desarrollada en Java 17 con interfaz gráfica JavaFX que permite la gestión integral de misiones espaciales. El sistema permite registrar, consultar, modificar y eliminar misiones, astronautas, vehículos de lanzamiento, satélites, registros de telemetría y estaciones de seguimiento terrestre.

La persistencia de datos se implementa mediante ObjectDB, una base de datos orientada a objetos embebida, accedida a través de la API estándar JPA 2.2. Esto permite trabajar directamente con objetos Java anotados con `@Entity` sin necesidad de un servidor de base de datos externo.

El proyecto ha sido desarrollado de forma colaborativa por un equipo de cinco personas durante el segundo trimestre del curso 2025–2026, siguiendo buenas prácticas de control de versiones con Git y GitHub, y aplicando una arquitectura en tres capas: UI (JavaFX) → Servicio (lógica de negocio) → Repositorio (acceso JPA).

2. Distribución de roles

Nombre	Rol	Tareas principales	Dedicación
Javier Fernández	Jefe de Proyecto	Coordinación del equipo, seguimiento de plazos, revisión final del entregable y gestión del repositorio GitHub.	~20 %
Héctor Rodríguez	Diseñador de BD	Modelo E-R, definición de entidades JPA, restricciones de integridad, script SQL de referencia y persistence.xml.	~20 %
Carlos Fernández	Desarrollador Backend	Implementación de repositorios, servicios y validaciones de negocio (RS-001 a RS-005). Integración JPA–ObjectDB.	~25 %
Antonio Manuel Rodríguez	Desarrollador de Interfaz	Diseño e implementación de vistas FXML, controladores JavaFX y flujo de navegación de la aplicación.	~25 %
Rafa Navarro	Calidad y Documentación	Redacción de la memoria, revisión de código, mantenimiento del README y elaboración de los diagramas.	~10 %

Cada miembro del equipo asumió su rol de forma principal, pero colaboró en tareas de otras áreas cuando fue necesario, especialmente en las fases de integración y pruebas finales.

3. Diagrama Entidad-Relación

3.1 Diagrama conceptual

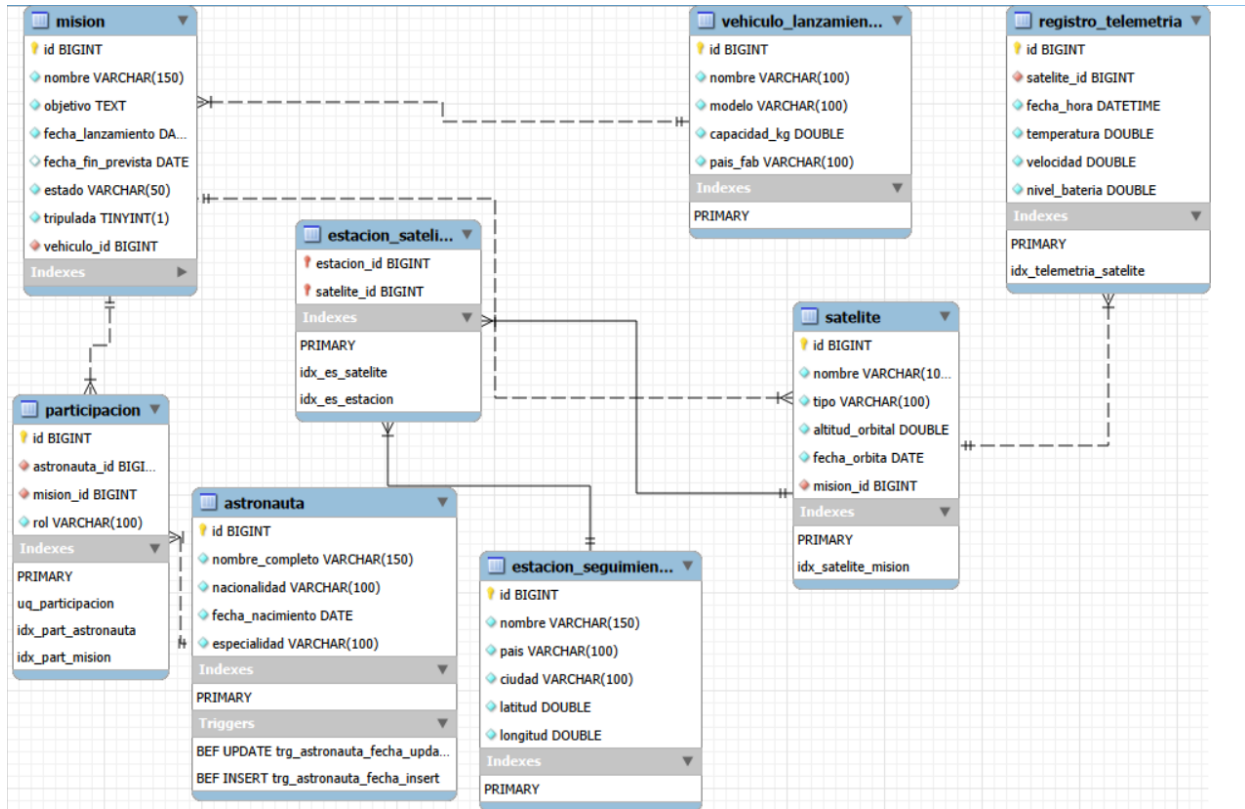


Figura 1. Diagrama E-R del sistema de gestión de misiones espaciales.

3.2 Descripción de entidades y relaciones

Entidad	Atributos clave	Relaciones
Mision	nombre (único), objetivo, fechaLanzamiento, fechaFinPrevista, estado, tripulada	N:1 VehiculoLanzamiento · 1:N Satellite · 1:N Participacion
VehiculoLanzamiento	nombre, modelo, capacidadKg, paisFab	1:N Mision
Astronauta	nombreCompleto, nacionalidad, fechaNacimiento, especialidad	1:N Participacion
Participacion	rol	N:1 Astronauta · N:1 Mision (entidad asociativa)
Satellite	nombre, tipo, altitudOrbital, fechaOrbita	N:1 Mision · N:M EstacionSeguimiento · 1:N RegistroTelemetria
RegistroTelemetria	fechaHora, temperatura, velocidad, nivelBateria	N:1 Satellite (entidad débil: acceso solo por ID)

EstacionSeguimiento	nombre, pais, ciudad, latitud, longitud	N:M Satelite
---------------------	---	--------------

3.3 Decisiones de diseño

Entidad asociativa Participacion

La relación entre Astronauta y Mision no se modela como @ManyToMany directo, sino a través de la entidad Participacion. Esto permite almacenar el atributo rol, que describe la función concreta de cada astronauta en cada misión (comandante, piloto, especialista de carga, etc.).

Entidad débil RegistroTelemetry

RegistroTelemetry se trata como entidad débil: dado el volumen masivo de lecturas que genera cada satélite, el sistema no ofrece listado global de todos los registros (RF-003 excluye explícitamente a esta entidad). El acceso solo es posible conociendo el ID del registro (RF-002) o a través del satélite al que pertenece (RF-007). Esto condiciona tanto el diseño del modelo como la implementación de los repositorios.

Relación N:M EstacionSeguimiento — Satelite

Esta relación se implementa con @ManyToMany y una tabla de unión estacion_satelite. Una misma estación puede rastrear múltiples satélites y un satélite puede ser monitoreado por varias estaciones simultáneamente.

Unicidad de nombre de misión

El campo nombre de Mision está marcado como @Column(unique = true) a nivel JPA y como UNIQUE INDEX en el script SQL de referencia. La validación RS-005 también se aplica en la capa de servicio antes de intentar la inserción.

4. Diagrama de clases

4.1 Descripción de la arquitectura

El proyecto sigue una arquitectura en tres capas con separación clara de responsabilidades:

- Capa UI (paquete ui): controladores JavaFX que leen la entrada del usuario, llaman a los servicios y actualizan las vistas FXML. Ningún controlador accede directamente a un repositorio.
- Capa Service (paquete service): clases de servicio que aplican las validaciones de negocio (RS-001 a RS-005) antes de delegar en los repositorios. Lanza `IllegalArgumentException` con mensajes descriptivos.
- Capa Repository (paquete repository): clases de acceso a datos que utilizan `EntityManager` de JPA para realizar operaciones CRUD y consultas JPQL. Cada entidad tiene su propio repositorio.
- Capa Modelo (paquete model): entidades JPA anotadas con `@Entity`, `@Id`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`, etc.
- Utilidades (paquete util): `JPAUtil` implementa el patrón Singleton para gestionar el `EntityManagerFactory`, que es costoso de crear y debe instanciarse una sola vez.

4.2 Clases principales

Entidades (model)

Mision · VehiculoLanzamiento · Astronauta · Participacion · Satelite · RegistroTelemetria · EstacionSeguimiento. Todas implementan `Serializable` (requerido por ObjectDB) y siguen el patrón `JavaBean` con constructor vacío, constructor completo, getters y setters.

Repositorios (repository)

MisionRepository · VehiculoRepository · AstronautaRepository · ParticipacionRepository · SateliteRepository · TelemetriaRepository · EstacionRepository. Cada uno gestiona el ciclo de vida del `EntityManager` (begin / commit / rollback / close) de forma independiente por operación.

Servicios (service)

MisionService (RS-001, RS-005) · AstronautaService (RS-004) · SateliteService (RS-002) · TelemetriaService (RS-003) · VehiculoService · EstacionService. Actúan como único punto de entrada para la capa UI.

Controladores UI (ui)

App (JavaFX Application) · MainController · MisionController · AstronautaController · SateliteController · TelemetriaController · VehiculoController · EstacionController. Cada controlador tiene su vista FXML asociada en resources.

Utilidades (util)

`JPAUtil`: singleton que gestiona el `EntityManagerFactory` para la unidad de persistencia `agencia-espacial` definida en `persistence.xml`. Se libera al cerrar la aplicación (`App.stop()`).

5. Justificación de tecnologías

Tecnología	Versión	Uso en el proyecto	Motivo de elección
Java	17 LTS	Lenguaje principal de toda la aplicación	Madurez, tipado estático, ecosistema Maven y soporte nativo de JPA
JavaFX	17.0.11	Interfaz gráfica (FXML + Controllers)	Integración nativa con Java, separación vista/lógica vía FXML
ObjectDB	2.9.0	Base de datos orientada a objetos embebida	Sin servidor externo, persistencia directa de objetos Java
JPA 2.2	2.2	API de persistencia (anotaciones, JPQL, EntityManager)	Abstracción del motor de BD, portabilidad del código de acceso a datos
Apache Maven	3.8+	Gestión de dependencias y compilación	Estandariza el build, facilita la incorporación de ObjectDB y JavaFX
IntelliJ IDEA	2024+	IDE de desarrollo	Soporte excelente para Java, Maven, JavaFX y control de versiones Git

5.1 Java 17

Java es el lenguaje natural para este tipo de aplicación de escritorio empresarial por varias razones que el equipo ha podido comprobar de forma directa durante el desarrollo:

- Tipado estático: los errores de tipo se detectan en compilación, no en ejecución. Esto resultó especialmente útil al definir las relaciones entre entidades JPA, donde un tipo incorrecto en un campo de relación produce errores difíciles de rastrear en tiempo de ejecución.
- Interoperabilidad con JPA: las anotaciones `@Entity`, `@ManyToOne`, `@OneToMany`, etc., son parte del estándar Java EE / Jakarta EE y funcionan de forma transparente con ObjectDB sin ninguna configuración adicional.
- Java 17 LTS: elegimos la versión 17 (LTS) por su compatibilidad garantizada con las versiones de JavaFX y ObjectDB utilizadas, y por ser la versión estable de referencia en el ciclo de vida largo de soporte.

Dificultades encontradas: la configuración del `module-path` de JavaFX con Java 9+ introduce complejidad adicional. La solución fue delegar la configuración en el plugin `javafx-maven-plugin`, que gestiona automáticamente las opciones `--module-path` y `--add-modules`.

5.2 JavaFX 17

JavaFX se eligió como framework de interfaz gráfica por su separación clara entre vista (FXML) y lógica de presentación (Controller), que se alinea con el patrón MVC que el equipo ya conocía de otros proyectos.

- FXML: permite diseñar la interfaz en XML de forma declarativa, separando completamente la estructura visual del código Java. Esto facilitó que el desarrollador de interfaz trabajara en paralelo con los desarrolladores de backend.
- TableView: la gestión de listas en TableView con ObservableList y PropertyValueFactory simplificó enormemente la visualización de colecciones de entidades.
- FXMLLoader: la carga dinámica de vistas mediante FXMLLoader.load() permitió estructurar la aplicación en ventanas independientes por entidad, mejorando la organización del código.

Dificultades encontradas: la configuración de JavaFX como módulo en Maven requiere declarar explícitamente las dependencias javafx-controls y javafx-fxml, y añadir el plugin javafx-maven-plugin. Inicialmente la aplicación fallaba con un NoClassDefFoundError de javafx.application.Application hasta que se configuró correctamente el classpath.

5.3 ObjectDB y JPA 2.2

La elección de una base de datos orientada a objetos frente a una relacional fue una decisión central del proyecto que justificamos en detalle a continuación.

Base de datos orientada a objetos vs. relacional

Las bases de datos relacionales (MySQL, PostgreSQL, SQLite) almacenan los datos en tablas con filas y columnas, y requieren un proceso de mapeo objeto-relacional (ORM) para traducir entre el modelo de objetos Java y el modelo tabular. Con JPA sobre una base relacional, aunque las anotaciones abstraen gran parte de este mapeo, sigue siendo necesario gestionar la creación de tablas (DDL), las migraciones de esquema y las inconsistencias de impedancia entre tipos Java y tipos SQL.

ObjectDB, al ser una base de datos orientada a objetos, almacena directamente los objetos Java serializados. Esto elimina completamente la impedancia objeto-relacional: un objeto Misión se guarda tal cual, con todas sus relaciones, sin necesidad de mapeo a tablas. Las ventajas que hemos experimentado directamente son:

- Configuración mínima: no hay servidor que instalar ni esquema que crear. El fichero agencia-espacial.odb se genera automáticamente al primer arranque.
- Persistencia transparente: las anotaciones JPA funcionan de forma idéntica a como lo harían sobre Hibernate + MySQL, pero sin necesidad de configurar un DataSource ni un servidor.
- JPQL nativo: ObjectDB implementa completamente JPQL, incluyendo joins, subconsultas y funciones de agregación, lo que nos permitió escribir las consultas de relaciones complejas (satélites por misión, misiones por estación) de forma estándar.

Como desventaja, ObjectDB tiene una comunidad más pequeña que los ORM relacionales como Hibernate, y algunos mensajes de error son menos descriptivos. En ocasiones fue necesario consultar la documentación oficial de ObjectDB para resolver comportamientos específicos de su motor (por ejemplo, la obligatoriedad de registrar explícitamente las clases en persistence.xml).

5.4 Apache Maven

Maven se eligió como herramienta de gestión de dependencias y compilación por las siguientes razones:

- Resolución automática de dependencias: ObjectDB está disponible en su repositorio Maven oficial (m2.objectdb.com), lo que permitió añadirlo al proyecto con una única entrada en pom.xml sin descargas manuales.
- Plugins de JavaFX: el plugin javafx-maven-plugin gestiona automáticamente el module-path de JavaFX, evitando la configuración manual de las opciones de la JVM.

- Portabilidad: cualquier miembro del equipo puede clonar el repositorio y ejecutar `mvn javafx:run` sin configurar nada más, lo que simplificó enormemente la incorporación de nuevos miembros y la resolución de incidencias en distintos equipos.

Dificultades encontradas: la primera vez que se configuró el repositorio de ObjectDB en `pom.xml`, Maven tardó varios minutos en resolver las dependencias por la latencia del repositorio externo. Se solucionó añadiendo el repositorio al `settings.xml` local para que se cacheara correctamente.

5.5 IntelliJ IDEA

IntelliJ IDEA fue el IDE elegido por unanimidad del equipo por su soporte completo para todos los lenguajes y tecnologías del proyecto: Java, JavaFX (con editor visual de FXML), Maven, JPA (con inspecciones de anotaciones) y Git (cliente integrado con historial gráfico de ramas).

- El inspector de JPA de IntelliJ detectó errores en las anotaciones de relaciones antes de ejecutar el código, ahorrando tiempo de depuración.
- El editor de FXML permite alternar entre la vista XML y el SceneBuilder integrado, facilitando el diseño de las vistas.
- La integración Git de IntelliJ facilitó la gestión de ramas y la resolución de conflictos de merge.

6. Estrategia de Git

6.1 Modelo de ramas

El equipo adoptó un modelo de ramas basado en Feature Branches, donde cada desarrollador trabaja en su rama de funcionalidad independiente y la integra en main mediante Pull Request revisada por al menos otro miembro del equipo.

Rama	Responsable	Propósito
main	Javier Fernández	Rama principal. Siempre contiene una versión funcional del proyecto. Solo se integran cambios mediante Pull Request revisada.
feature/model	Héctor Rodríguez	Desarrollo de las entidades JPA (@Entity, relaciones, persistence.xml) y script SQL de referencia.
feature/repository	Carlos Fernández	Implementación de los repositorios (CRUD con EntityManager) y la clase utilitaria JPAUtil.
feature/service	Carlos Fernández	Capa de lógica de negocio: servicios con validaciones RS-001 a RS-005.
feature/ui	Antonio Manuel Rodríguez	Vistas FXML y controladores JavaFX para las seis entidades principales.
feature/docs	Rafa Navarro	README.md, memoria del proyecto y diagramas.

6.2 Flujo de trabajo

El flujo de trabajo seguido en cada iteración fue el siguiente:

1. Creación de la rama de funcionalidad a partir de main: `git checkout -b feature/nombre`.
2. Desarrollo incremental con commits frecuentes y mensajes descriptivos en español.
3. Push de la rama al repositorio remoto y apertura de Pull Request en GitHub.
4. Revisión del código por otro miembro del equipo (mínimo uno), con comentarios en la PR.
5. Merge a main tras aprobación, usando squash merge para mantener el historial limpio.
6. Eliminación de la rama de funcionalidad tras el merge.

6.3 Convención de mensajes de commit

Los mensajes de commit siguen el formato: [tipo]: descripción breve en español. Los tipos utilizados son:

- feat: nueva funcionalidad (ej. feat: añadir repositorio de misiones).
- fix: corrección de errores (ej. fix: corregir validación RS-004 en AstronautaService).
- docs: documentación (ej. docs: actualizar README con instrucciones de ejecución).
- refactor: reorganización de código sin cambio de funcionalidad.
- test: añadir o modificar pruebas.

6.4 Problemas encontrados y soluciones

Conflicto de merge en persistence.xml

Al integrar las ramas feature/model y feature/repository, se produjo un conflicto en el fichero persistence.xml porque ambas ramas lo habían modificado. Se resolvió manualmente fusionando las entidades registradas de ambas versiones. A partir de entonces, se estableció como norma que cualquier modificación de persistence.xml se coordinase con el responsable de la rama feature/model.

Rama feature/ui retrasada respecto a main

La rama de interfaz estuvo trabajando durante varios días sin sincronizar con main, lo que produjo un merge con múltiples conflictos en las clases de servicio cuando los servicios fueron refactorizados en la rama principal. La solución fue hacer git rebase main desde feature/ui para rebasar los cambios de la interfaz sobre la versión más reciente de main antes del merge final.

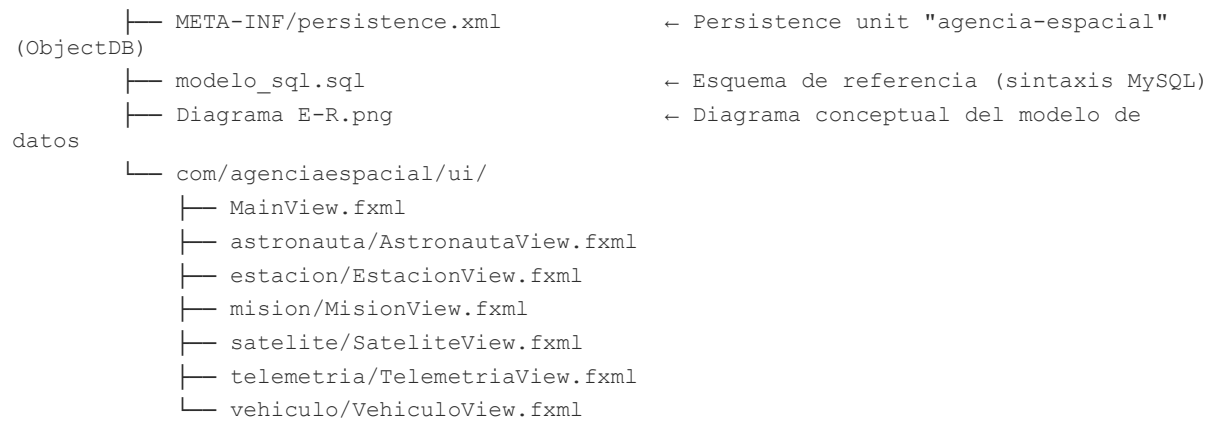
7. Estructura del proyecto

A continuación, se describe la organización del código fuente y el propósito de cada carpeta y archivo relevante:

```

agencia-espacial/
├── pom.xml                ← Configuración Maven (Java 17, ObjectDB
2.9.0, JavaFX 17)
├── .gitignore            ← Excluye target/, data/, *.odb,
archivos de IDE
└── src/main/
    ├── java/com/agenciaespacial/
    │   ├── Main.java      ← Punto de entrada · delega en App.java
    │   ├── model/
    │   │   ├── Mision.java ← @Entity · misión espacial (tripulada o
no)
    │   │   ├── Astronauta.java ← @Entity · participante en misiones
tripuladas
    │   │   ├── Participacion.java ← @Entity · relación Astronauta ↔ Mision
con rol
    │   │   ├── VehiculoLanzamiento.java ← @Entity · vehículo usado en una misión
    │   │   ├── Satelite.java ← @Entity · satélite puesto en órbita
    │   │   ├── RegistroTelemetria.java ← @Entity · lectura de telemetría de un
satélite
    │   │   └── EstacionSeguimiento.java ← @Entity · estación terrestre de
monitorización
    │   ├── repository/
    │   │   ├── MisionRepository.java
    │   │   ├── AstronautaRepository.java
    │   │   ├── ParticipacionRepository.java
    │   │   ├── VehiculoRepository.java
    │   │   ├── SateliteRepository.java
    │   │   ├── TelemetriaRepository.java
    │   │   └── EstacionRepository.java
    │   ├── service/
    │   │   ├── MisionService.java ← Aplica RS-001 y RS-005
    │   │   ├── AstronautaService.java ← Aplica RS-004
    │   │   ├── SateliteService.java ← Aplica RS-002
    │   │   ├── TelemetriaService.java ← Aplica RS-003
    │   │   ├── VehiculoService.java
    │   │   └── EstacionService.java
    │   └── ui/
    │       ├── App.java ← Application JavaFX · carga
MainView.fxml
    │       ├── MainController.java ← Abre las ventanas secundarias
    │       ├── mision/MisionController.java
    │       ├── astronauta/AstronautaController.java
    │       ├── satelite/SateliteController.java
    │       ├── telemetria/TelemetriaController.java
    │       ├── vehiculo/VehiculoController.java
    │       └── estacion/EstacionController.java
    └── util/
        ├── JPAUtil.java ← Singleton · gestiona
EntityManagerFactory
    └── resources/

```



7.1 Descripción de los paquetes

- **model:** contiene las siete entidades JPA del sistema, cada una anotada con `@Entity` y con sus relaciones declaradas mediante las anotaciones de JPA.
- **repository:** acceso a datos. Cada repositorio gestiona el ciclo de vida del `EntityManager` de forma independiente por operación, siguiendo el patrón `open-session-per-request`.
- **service:** lógica de negocio. Es el único punto de acceso a datos para la capa de presentación. Aplica las validaciones RS-001 a RS-005 antes de delegar en el repositorio.
- **ui:** controladores JavaFX. Cada controlador carga su vista FXML desde el directorio `resources` y se comunica exclusivamente con la capa `service`. Las vistas se abren como `Stage` independientes.
- **util:** `JPAUtil` implementa el `Singleton` del `EntityManagerFactory`. Es compartido por todos los repositorios para que solo exista una instancia de la factoría durante todo el ciclo de vida de la aplicación.
- **resources:** ficheros de configuración JPA (`persistence.xml`), vistas FXML, el diagrama E-R en PNG y el script SQL de referencia generado con MySQL Workbench.